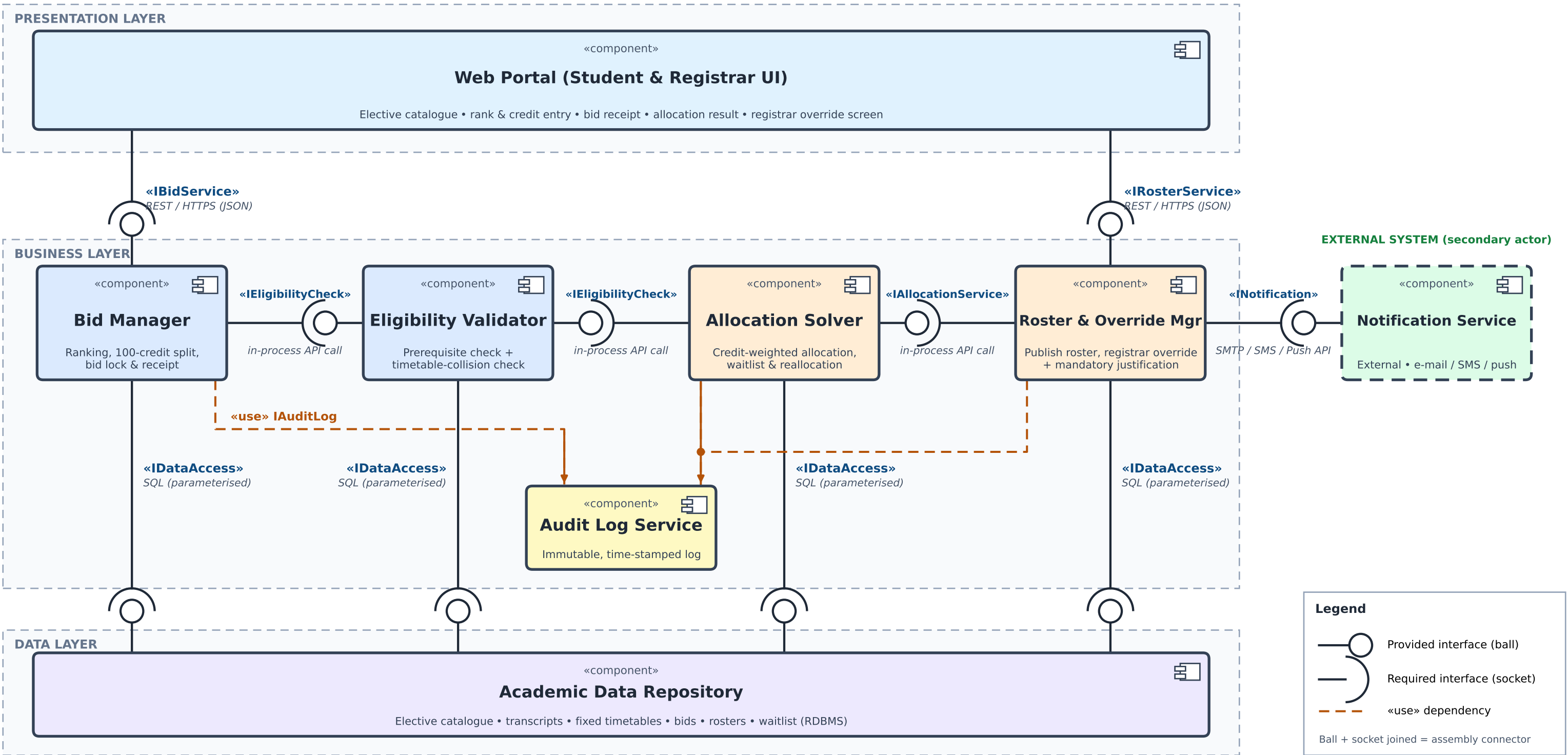


UML Component Diagram — Academic Elective Bidding & Allocation System

Layered architecture (Presentation → Business → Data) | Lab 3 | PES1UG24AM446



Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification | SRN: PES1UG24AM446 | System: Academic Elective Bidding & Allocation System (Problem Statement #05, from Lab 1)

Architecture Selection

We chose **Layered Architecture (Presentation, Business, Data)**, deployed as a **client-server web application**, for the Academic Elective Bidding & Allocation System.

Style	Strength for this system	Weakness for this system	Verdict
Layered	Clear UI / logic / data separation; one transactional database; in-process calls	Harder to scale single layers independently	Selected
Microservices	Independent scaling and deployment	Distributed data consistency, network latency, heavy ops overhead	Rejected
Client-Server	Simple deployment, central control	Single point of failure; no internal structure for 8 components	Used only as the deployment model

Two Reasons for This Choice

1. Credit and seat integrity needs one consistent data store. FR-001 and FR-002 require exactly 100 credits with unique ranks, FR-003 requires collision checks against the fixed timetable, and FR-004 reassigns freed seats. All of these must read and update the same bids, timetable and seat counts atomically. A single Data layer gives ACID transactions. Microservices would split this state and need distributed transactions, with seat-count races when the bidding window closes.

2. The components map directly onto the Lab 1 use cases and stay replaceable. Submit Elective Bid becomes Bid Manager. Validate Prerequisite Completion and Detect Timetable Collision become Eligibility Validator. Run Allocation Solver becomes Allocation Solver. Publish Roster and Override Manual Allocation become Roster & Override Manager. Notification and audit logging become the Notification and Audit Log services. Each is reached only through a defined interface (IBidService, IEligibilityCheck, IAllocationService and so on), so the solver algorithm can be swapped without touching the UI. A department-scale system does not justify the operational complexity of microservices.

Security Advantage

The layers act as controlled checkpoints. The Web Portal never reaches the database; only the Business layer calls IDataAccess, using parameterised queries, which blocks direct access and SQL injection from the UI. Authentication and role checks (Student versus Registrar) are enforced once at the service boundary, so a registrar override is accepted only through IRosterService with a mandatory justification note (FR-005). Every bid, allocation change and override is written through IAuditLog to an append-only, time-stamped Audit Log Service, which satisfies NFR-002. Traffic between browser and server uses HTTPS.

Performance Benefit

The Eligibility Validator, Allocation Solver and Roster Manager run in the same Business-layer process and call each other in-process. There are no network hops between services, unlike microservices, so prerequisite and timetable checks for each of the 5,000 bids cost microseconds rather than round-trips. The solver loads all bids with set-based queries through IDataAccess and solves in memory as a one-time batch after the window closes, which supports the 30-second target in NFR-001. Catalogue and timetable data can be cached in the Data-access layer. The layered weakness of scaling is handled by running the stateless Portal and Business tiers on several servers behind a load balancer during the bidding peak.

Components and Interfaces (see diagram)

8 components: Web Portal, Bid Manager, Eligibility Validator, Allocation Solver, Roster & Override Manager, Audit Log Service, Notification Service (external), Academic Data Repository. **7 interfaces:** IBidService, IRosterService, IEligibilityCheck, IAllocationService, IDataAccess, INotification (ball and socket) and IAuditLog («use» dependency).